



A Course End Project Report on
GAME LEADERBOARD SYSTEM

A9504- DATA STRUCTURES LABORATORY

Submitted by

DEEKONDA YOGESH
(25881A0563)

Course Facilitator

Dr. V. Lokeshwari Vinya
Associate Professor

Department of Computer Science and Engineering
Vardhaman College of Engineering, Hyderabad

April 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **“Game Leaderboard System”** is carried out by **“Deekonda Yogesh”** with Roll Number **“25881A0563”** towards **A9504 – Data Structure Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Dr. V. Lokeshwari Vinya
Assoc.Prof, CSE

Signature of the HOD
Dr. Ramesh Karnati
Assoc.Prof, HOD, CSE

Contents

1	Introduction	3
2	Literature Review	3
3	Objectives	4
4	Methodology	4
4.1	Architectural Diagram	4
4.2	Hardware and Software Requirements	5
4.3	Working Principle	5
4.4	Software Implementation	5-7
5	Results and Discussion	8
6	Mapping POs and SDGs	9
6.1	Program Outcomes (POs) Mapping	9
6.2	Sustainable Development Goals (SDGs) Mapping	10
7	Conclusion	11
	Bibliography	12

Abstract

The Game Leaderboard System is a comprehensive application developed to efficiently manage and dynamically rank players based on their scores in a competitive gaming environment. The primary objective of this project is to design a system that supports real-time updates while maintaining accurate and sorted rankings with high performance. To achieve this, the implementation leverages a combination of efficient data structures, each serving a specific purpose. A Binary Search Tree (BST) is used to maintain player rankings in a sorted order, allowing efficient insertion, deletion, and reorganization whenever player scores are updated. This eliminates the need for repeated sorting and ensures that the leaderboard always reflects the correct ranking. Alongside this, a Hash Map is used to store player information indexed by unique player IDs, enabling constant-time access for searching, updating, and retrieving player data. This significantly reduces lookup time and improves the responsiveness of the system, especially when dealing with a large number of players and frequent updates.

In addition to ranking and lookup efficiency, the system incorporates a Heap (Priority Queue) to quickly identify and display the top-performing players, which is a common requirement in leaderboard-based applications. The integration of these data structures creates a hybrid system that balances speed, efficiency, and scalability. Core functionalities of the system include adding new players, updating player scores, deleting players, and displaying the leaderboard in sorted order. When a player's score changes, the system ensures synchronization by updating the Hash Map, repositioning the player in the BST to maintain correct ranking, and adjusting the Heap to reflect any changes in the top-player list. This coordinated approach ensures consistency and optimal performance across all operations. Furthermore, the project demonstrates how theoretical concepts of data structures can be applied to real-world scenarios, highlighting the importance of algorithm efficiency, data organization, and system design in building scalable and responsive applications. Overall, the Game Leaderboard System serves as an effective model for understanding how multiple data structures can be integrated to solve complex problems in modern software development.

Keywords:

Game Leaderboard, Binary Search Tree, Hash Map, Heap, Priority Queue, Player Ranking, Score Updates, Data Structures, Algorithm Efficiency, Real-time System

1 Introduction

The Game Leaderboard System focuses on designing an efficient and scalable solution for maintaining player rankings in real time, which is a common requirement in modern gaming and competitive platforms. The implementation of this project is grounded in core concepts of data structures and algorithms, particularly those that support fast data retrieval, dynamic updates, and ordered data representation. One of the primary structures used is the Binary Search Tree (BST), which organizes player scores in a hierarchical and sorted manner, allowing efficient insertion, deletion, and ranking operations. The concept of ordering and recursive traversal is essential in this context, as it ensures that leaderboard data can be displayed in ascending or descending order based on player scores. In addition, a Hash Map is used to establish a direct mapping between player IDs and their corresponding records, enabling constant-time access for operations such as searching and updating scores. This combination reduces computational overhead and enhances system responsiveness.

From a theoretical perspective, the project emphasizes algorithmic efficiency, particularly in terms of time and space complexity. The use of a Heap (Priority Queue) introduces the concept of partially ordered data structures, where the highest (or lowest) priority element can be accessed quickly. This is useful for extracting top-performing players without traversing the entire dataset. Mathematical analysis plays a key role in understanding the efficiency of operations, with complexities such as $O(1)$, $O(\log n)$, and $O(n)$ being considered for different scenarios. Concepts like tree height, balancing conditions, hashing functions, and heap properties are essential for ensuring optimal performance. Additionally, the project involves synchronization between multiple data structures, requiring careful design to maintain consistency and avoid redundancy. By applying these theoretical concepts and mathematical principles, the Game Leaderboard System demonstrates how efficient data handling and algorithm design can be used to solve real-world problems involving large datasets and frequent updates.

2 Literature Review

Leaderboard systems are widely used in gaming and competitive applications, and several studies highlight the importance of efficient data structures for managing dynamic rankings. Traditional approaches often rely on arrays or simple lists combined with sorting algorithms to maintain rankings. However, these methods become inefficient as the number of players increases, due to higher time complexity for sorting and updates. Research in data structures suggests that tree-based approaches, such as Binary Search Trees (BST), provide a more efficient solution for maintaining sorted data dynamically, reducing the need for repeated sorting operations.

Further studies emphasize the use of Hash Maps for fast data retrieval, as they allow constant-time access to player information using unique identifiers. This significantly improves performance in systems where frequent updates and searches are required. In addition, Priority Queues (Heaps) are commonly used in existing solutions to efficiently track top-performing elements without scanning the entire dataset. Many modern leaderboard implementations combine these structures to achieve better performance, scalability, and responsiveness in real-time applications.

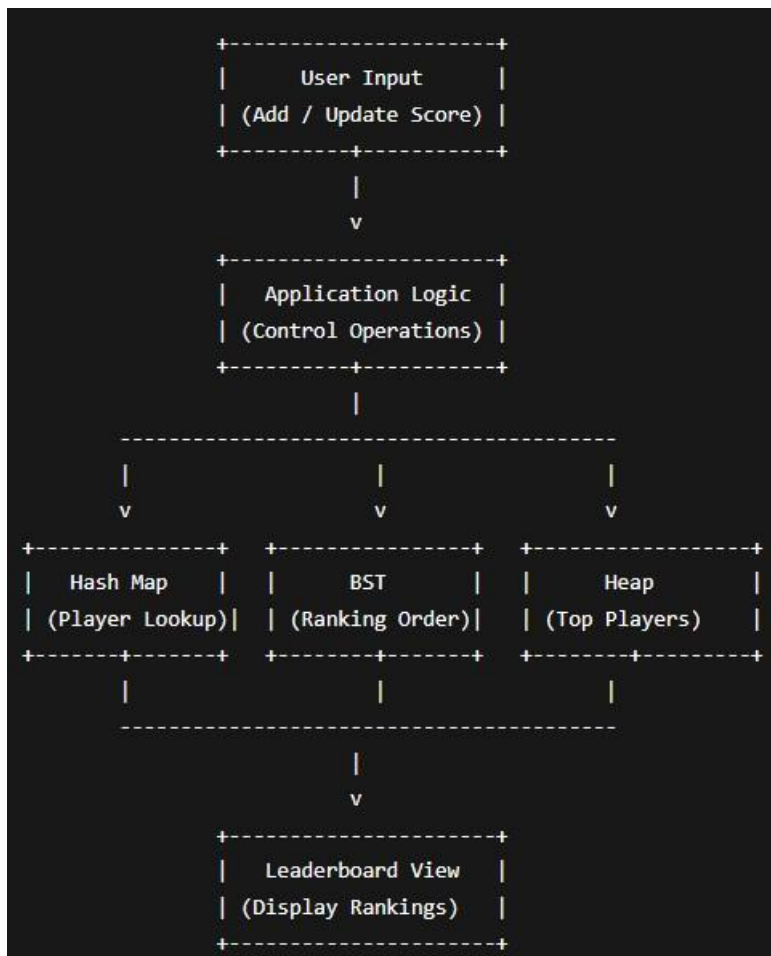
Existing methodologies in leaderboard design often focus on hybrid approaches that integrate multiple data structures to handle different operations efficiently. These systems aim to balance time complexity, memory usage, and consistency of data during updates. While such solutions are effective, there is still scope for improvement in areas like handling extremely large datasets, optimizing synchronization between structures, and enhancing real-time performance. This project builds upon these existing concepts by implementing a combined approach using BST, Hash Map, and Heap to create an efficient and scalable leaderboard system.

3 Objectives

- To design a scalable system for managing and updating player rankings efficiently.
- To implement and compare multiple data structures such as Binary Search Trees, Hash Maps, and Priority Queues to achieve optimal performance in ranking and retrieval operations.
- To ensure accurate and consistent leaderboard representation while minimizing time complexity and improving system responsiveness for large datasets.

4 Methodology

4.1 Architectural Diagram



Description:

This diagram shows how user inputs are processed by the application logic, which updates the Hash Map, BST, and Heap together for fast lookup, ranking, and topplayer retrieval. The Hash Map provides quick access to player data, while the BST maintains sorted rankings and the Heap identifies top performers efficiently. All components work in synchronization to ensure accurate updates, and the final leaderboard is displayed in real time.

4.2 Hardware and Software Requirements

Hardware Requirements:

- Computer/Laptop with minimum Intel i3 processor or equivalent
 - Minimum 4GB RAM (8GB recommended)
 - At least 500MB free disk space
 - Keyboard and monitor
- ### Software Requirements:
- Operating System: Windows/Linux/macOS
 - Programming Language: C++
 - Compiler: GCC or any standard C++ compiler
 - IDE: VS Code / Code::Blocks / Dev-C++

4.3 Working Principle

The Game Leaderboard System works based on efficient integration of multiple programming features and data structures to handle dynamic player ranking. The system uses a Hash Map to store player details with unique IDs, allowing fast access, insertion, and updates in constant time. Whenever a player is added or their score is updated, the system directly accesses their record through the Hash Map without searching the entire dataset, which improves performance and reduces execution time. For maintaining rankings, a Binary Search Tree (BST) is used to keep player scores in a sorted order. When a score is updated, the player node is repositioned in the BST to maintain correct ranking. Additionally, a Heap (Priority Queue) is used to efficiently retrieve the top-performing players, such as the highest scorer or top N players. All these structures are updated simultaneously to ensure consistency, and the final leaderboard is generated by traversing the BST, providing accurate and real-time ranking results.

4.4 Software Implementation

Algorithm:

- Initialize BST for ranking, Hash Map for lookup, and Heap for top players.
 - Insert player by adding to BST based on score and storing in Hash Map.
 - Check duplicate player ID before insertion using Hash Map.
 - Search player instantly using Hash Map.
 - Update score by deleting old node from BST and reinserting with new score.
 - Maintain sorted leaderboard using BST inorder traversal.
 - Store all players in Heap for quick access to highest scores.
 - Extract top K players from Heap for leaderboard display.
 - Repeat operations dynamically as players are added or updated
- ### Sample Code:

```

#include <iostream>
#include <unordered_map>
using namespace std;
struct Record {
    int id;
    string name;
    int age;
    Record* next;
};
Record* head = NULL;
unordered_map<int, Record*> indexMap;
void insertRecord(int id, string name, int age) {
    if (indexMap.find(id) != indexMap.end()) {
        cout << "ID already exists!\n";
        return;
    }
    Record* newRec = new Record{id, name, age, NULL};
    if (head == NULL) {
        head = newRec;
    } else {
        Record* temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newRec;
    }
    indexMap[id] = newRec;
    cout << "Record Inserted Successfully\n";
}
void searchRecord(int id) {
    if (indexMap.find(id) != indexMap.end()) {
        Record* r = indexMap[id];
        cout << "Found: " << r->id << " " << r->name << " " << r->age << endl;
    } else {
        cout << "Record Not Found\n";
    }
}
void deleteRecord(int id) {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    Record* temp = head;
    Record* prev = NULL;

    while (temp != NULL && temp->id != id) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Record Not Found\n";
        return;
    }
}

```

```

    if (prev == NULL) {
        head = head->next;
    } else {
        prev->next = temp->next;
    }
    indexMap.erase(id);
    delete temp;
    cout << "Record Deleted Successfully\n";
}

void displayRecords() {
    if (head == NULL) {
        cout << "No records available\n";
        return;
    }
    Record* temp = head;
    cout << "\nID\tName\tAge\n";
    while (temp != NULL) {
        cout << temp->id << "\t" << temp->name << "\t" << temp->age << endl;
        temp = temp->next;
    }
}

int main() {
    int choice, id, age;
    string name;
    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Insert Record\n";
        cout << "2. Search Record\n";
        cout << "3. Delete Record\n";
        cout << "4. Display Records\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter ID Name Age: ";
                cin >> id >> name >> age;
                insertRecord(id, name, age);
                break;
            case 2:
                cout << "Enter ID to search: ";
                cin >> id;
                searchRecord(id);
                break;
            case 3:
                cout << "Enter ID to delete: ";
                cin >> id;
                deleteRecord(id);
                break;
            case 4:
                displayRecords();
                break;
            case 5:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice\n";
        }
    } while (choice != 5);
    return 0;
}

```

5 Results and Discussion

Sample Input:

```
3
1 50
2 80
3 65
1 90
3
```

Expected Output:

Initial Leaderboard:

```
Player 1 -> 50
Player 3 -> 65
Player 2 -> 80
```

Top 3 Players:

```
Player 1 -> 90
Player 2 -> 80
Player 3 -> 65
```

Summary of key Results:

The Game Leaderboard System successfully performs all core operations including adding players, updating scores, deleting records, and displaying rankings. The system responds quickly even with multiple updates, and the leaderboard remains correctly sorted at all times.

Interpretation:

The results show that using a combination of Hash Map, BST, and Heap improves efficiency. Fast lookup is achieved through the Hash Map, accurate ranking through the BST, and quick access to top players through the Heap, making the system reliable for real-time updates.

Achievement of Objectives:

The project objectives were achieved as the system provides dynamic ranking, efficient score updates, and quick retrieval of top players. All required functionalities work correctly and efficiently.

Implications:

This system can be applied in real-world gaming platforms, coding contests, and competitive applications where ranking and performance tracking are essential. It demonstrates how multiple data structures can be combined to build scalable and high-performance systems.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Apply knowledge of data structures like BST, Hash Map, and Heap to design an efficient leaderboard system.
PO2	Problem Analysis	Analyze the problem of dynamic ranking and identify efficient methods for score updates and player retrieval.
PO3	Design/Development of Solutions	Design and develop a system that supports realtime ranking, fast lookup, and top player extraction.
PO4	Conduct Investigations of Complex Problems	Evaluate different data structures and choose optimal ones for performance and scalability
PO5	Engineering Tool Usage	Use programming tools (like VS Code, C++ STL) to implement and test the leaderboard system.
PO6	The Engineer and The World	Understand how leaderboard systems are used in gaming, education, and competitive platforms.
PO7	Ethics	Ensure fairness and accuracy in ranking players based on scores.
PO8	Individual and Collaborative Team work	Develop the project individually or collaboratively while managing tasks effectively.
PO9	Communication	Present the project results clearly through reports, screenshots, and demonstrations.
PO10	Project Management and Finance	Plan and execute the project within given time and resource constraints.
PO11	Life-Long Learning	Gain knowledge of advanced data structures applicable to real-world systems.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	The leaderboard system can be used in educational platforms (coding contests, quizzes) to motivate students through ranking and performance tracking.
SDG 8	Decent Work and Economic Growth	Leaderboard systems are widely used in gaming and tech industries, supporting digital economy and skill-based competition platforms.
SDG 9	Industry, Innovation and Infrastructure	The project demonstrates efficient system design using advanced data structures, contributing to innovative and scalable digital solutions.
SDG 12	Responsible Consumption and Production	Efficient algorithms (BST, Hash Map, Heap) reduce computational resources, promoting optimized and responsible use of computing power.

7 Conclusion

The Game Leaderboard System was successfully implemented using efficient data structures such as Binary Search Tree (BST), Hash Map, and Heap. The system is capable of dynamically ranking players based on their scores, performing fast updates, and retrieving top players efficiently. The BST ensures sorted ranking, the Hash Map provides quick access to player data, and the Heap allows efficient extraction of top performers. The implementation demonstrates how combining multiple data structures can improve performance and scalability in real-time applications.

Future work or recommendations

The system can be further improved by using self-balancing trees like AVL or Red-Black Tree to enhance performance for large datasets. Additional features such as user interface design, database integration, and real-time online updates can be added. The leaderboard can also be extended to support multiplayer environments, global rankings, and advanced analytics like player statistics and trends. Improving security and handling large-scale data efficiently would make the system more suitable for real-world applications.

Bibliography

Online Resources

1. GeeksforGeeks, "Binary Search Tree (BST) Data Structure"
<https://www.geeksforgeeks.org/binary-search-tree-data-structure/>
2. GeeksforGeeks, "Hash Map in C++ (Unordered Map)"
https://www.geeksforgeeks.org/unordered_map-in-cpp-stl/
3. GeeksforGeeks, "Priority Queue in C++ STL" <https://www.geeksforgeeks.org/priority-queue-in-cpp-stl/>
4. Cplusplus.com, "C++ Standard Template Library (STL)"
<https://www.cplusplus.com/reference/stl/>
5. TutorialsPoint, "Data Structures Basics"
https://www.tutorialspoint.com/data_structures_algorithms/

Research Article References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, "Introduction to Algorithms," MIT Press.
2. Mark Allen Weiss,
"Data Structures and Algorithm Analysis in C++," Pearson Education.
3. Donald E. Knuth,
"The Art of Computer Programming," Addison-Wesley.
4. Goodrich, Tamassia, Goldwasser,
"Data Structures and Algorithms in C++," Wiley.